

Vision Day01 hw2 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.21

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제에선 이미지에 존재하는 빨간색, 파란색, 초록색 객체를 동시에 인식하고, 각 객체에 해당 색상의 바운딩 박스를 표시하는 것이었다. 입력 이미지에는 빨간 공, 파란 공, 초록색 블록이 있으며, 프로그램은 세 객체를 색상에 따라 구분해야 한다. 구현 언어는 C++를 선택했고 영상 처리는 OpenCV를 사용했다.

과제를 수행하기 위해 이미지 입력, 색 공간 변환, 색상 범위 지정, 마스크 생성, 노이즈 제거, 윤곽선 검출, 바운딩 박스 표시 과정이 필요했다. AI에게 과제의 전체적인 구현 순서와 C++ 코드 작성 방법 및 OpenCV 함수를 질문하였고 도움을 받았다.

2. AI 사용 부분

과제를 C++로 작성하기로 결정 한 후 이후 과제에 사용할 빨간 공, 파란 공, 초록 블록 사진을 제공하고 해당 이미지에 맞는 C++ 코드를 AI의 도움을 받아 구성하였다. AI는 OpenCV의 `cv::imread()`로 이미지를 읽고, `cv::GaussianBlur()`로 작은 노이즈를 완화한 다음 `cv::cvtColor()`를 이용해 BGR 이미지를 HSV로 변환하도록 코드를 작성하였다. 다음은 코드를 작성하며 사용한 기법을 간단히 설명하겠다.

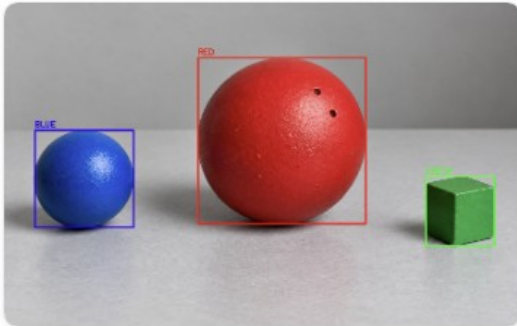
색상 분리에는 `cv::inRange()`를 사용하였다. 빨간색은 OpenCV의 Hue 범위 시작과 끝에 나뉘어 있기 때문에 (0, 90, 60)부터 (10, 255, 255)까지의 범위와 (170, 90, 60)부터 (179, 255, 255)까지의 범위를 각각 마스크로 만든 뒤 `cv::bitwise_or()`로 합쳤다. 파란색은 (100, 90, 50)부터 (115, 255, 255)까지, 초록색은 (45, 80, 30)부터 (75, 255, 255)까지의 범위를 사용하였다.

초록 블록은 빛을 받는 앞면과 어두운 옆면의 밝기 차이가 컸기 때문에 V의 하한을 비교적 낮게 설정하였다. 이를 통해 밝은 면과 어두운 면이 분리되지 않고 하나의 객체로 검출되도록 하였다. AI가 제안한 범위를 실제 결과 이미지에 적용한 결과 세 객체가 각각 하나의 박스로 검출되는 것을 볼 수 있었다.

마스크를 생성한 후에는 5×5 크기의 타원형 커널을 이용하여 Opening과 Closing 연산을 적용하였다. Opening은 객체 주변에 있는 작은 흰색 노이즈를 제거하기 위해 사용했고, Closing은 객체 내부에 생길 수 있는 작은 검은색 구멍을 메우기 위해 사용하였다. 이후 `cv::findContours()`로 마스크의 외곽 윤곽선을 찾고, 윤곽선의 면적이 2000보다 작은 경우에는 노이즈로 판단하여 제외하였다.

남은 윤곽선에는 `cv::boundingRect()`를 적용하여 객체를 둘러싸는 사각형을 구하였다. OpenCV에서는 색상 순서가 RGB가 아니라 BGR이기 때문에 빨간색 박스는 (0, 0, 255), 파란색 박스는 (255, 0, 0), 초록색 박스는 (0, 255, 0)으로 지정하였다. `cv::putText()`를 사용해 각 박스 위에 RED, BLUE, GREEN이라는 문자열도 표시하였다.

C++ 버전으로 완성했고, 제공한 사진에서 직접 검증했습니다. 빨강·파랑·초록이 각각 정확히 한 개의 박스로 검출됩니다.



파일:

-  main.cpp
-  CMakeLists.txt
-  result.png

실행 방법:

 Bash

```
cd assignment2_cpp
cmake -S . -B build
cmake --build build

./build/color_detector \
  "/home/doyeon/Documents/KakaoTalk
Downloads/vision/KakaoTalk_20260918_185908846.png" \
  result.png
```

사진에 맞춰 사용한 HSV 범위는 다음과 같습니다.

- 빨강: (0, 90, 60)~(10, 255, 255) 와 (170, 90, 60)~(179, 255, 255)
- 파랑: (100, 90, 50)~(115, 255, 255)
- 초록: (45, 80, 30)~(75, 255, 255)
- 최소 윤곽선 면적: 2000

코드 흐름은 GaussianBlur → BGR을 HSV로 변환 →  inRange → Opening/Closing →

채팅에 추가



```
assignment2_cpp/  
├── CMakeLists.txt  
├── main.cpp  
├── input.png  
└── result.png
```

main.cpp (line 51)에서 기본 경로가 다음처럼 상대경로로 설정돼 있습니다.

C++



```
const std::string inputPath = argc >= 2 ? argv[1] : "input.png";  
const std::string outputPath = argc >= 3 ? argv[2] : "result.png";
```

화면에서 안 된 이유는 터미널이 홈 폴더 `~` 에 있었기 때문입니다. `cd assignment2_cpp` 가 실패한 뒤에도 다음 명령들이 실행돼서:

- 홈 폴더에는 `CMakeLists.txt` 가 없음
- 빌드가 안 됐으므로 `build/color_detector` 도 없음

이라는 오류가 연달아 발생했습니다.

아래 명령을 그대로 한 줄씩 실행하세요.

Bash



```
cd "/home/doyeon/.codex/.chatgpt-projects/g-p-  
6aad1e7f94888191806c33fc29d8438b/assignment2_cpp"
```

Bash



```
cmake -S . -B build
```

Bash



```
cmake --build build
```



ChatGPT로 Work 시작

3. 깨달은 점

이번 과제를 하면서 HSV 를 사용하면 BGR 보다 색상을 구분하기 편하다는 것을 알게 되었다. HSV 는 색상, 채도, 밝기가 나뉘어 있어서 H 값으로 원하는 색상을 정하고, S 와 V 값으로 회색 배경이나 어두운 부분을 제외할 수 있었다. 다만 조명이나 그림자가 달라지면 같은 물체도 HSV 값이 달라질 수 있기 때문에 이미지에 맞게 범위를 직접 조절해야 한다는 것도 알게 되었다.

빨간색은 OpenCV 의 Hue 범위 양쪽에 나뉘어 있어서 두 개의 마스크를 만든 뒤 합쳐야 한다는 점이 신기했다. 또한 Opening 은 작은 흰색 노이즈를 제거하고, Closing 은 객체 안의 작은 구멍을 메우는 데 사용된다는 것을 실제 결과를 보면서 이해할 수 있었다.

AI 가 작성한 코드를 그대로 사용하는 것보다 각 함수가 어떤 역할을 하는지 확인하고 직접 해보는 과정이 중요하다고 느꼈다. 앞으로 비슷한 과제를 할 때도 전체 처리 순서를 먼저 정리하고, 코드를 실행하면서 하나씩 이해하는 방식으로 진행하고 싶다.